

10. Tuning of autoISF settings for Full Closed Loop aided by the Emulator

V.4.2

Please note that with autoISF you are in an early-dev. environment, where the user interface is **not optimized for safety** of users who stray away from intended ways to use. Good safety features exist, but these are only as good as the development-oriented user understands and implements them. This is not a medical product, refer to disclaimer in [section 0](#)



10.1 Installing the Emulator on your PC

10.1.1 File structure on your PC

10.1.2 Load config and py files

10.1.3 Desktop button "Emulation_start"

10.1.4 Other software requirements

10.2 Analyzing **loop decisions** in logfiles

10.2.1 **noChange.vdf**

10.2.2/3 Locate logfiles / prepare Emulator

10.2.4 Run emulation and inspect results

10.2.4.1 .txt (all AUTO ISF tab infos)

10.2.4.2 Tabular (.csv) presentation of all loop decisions

10.2.4.3 Manual extraction of key data into .xls or .odc

10.2.4.4 .pdf chart

10.2.4.5 delta table

10.3 What-if analysis

10.3.1 Define (**yourChange**).vdf

10.3.2 Run emulation

10.3.3 Inspect results

10.3.3.1 Logs (all AUTO ISF tab infos)

10.3.3.2 Tabular (.csv) presentation of all loop decisions

10.3.3.3 Semi-automated extraction of key data

10.3.3.4 .pdf chart

10.3.3.5 delta table

Available related case studies:

Case studies **still missing:**

Based on older autoISF and older emulator versions, examples from emulator use can be found in [case study 6.2](#), in [case study 4.1](#) (last pages there), and [case study 8.2](#)

You can set up and tune the system for Full Closed Loop as described in previous sections.

Doing this by

- frequently analyzing screenshots that must be taken in real-time of the AAPS **AUTO ISF tab**

39 is tedious, however.

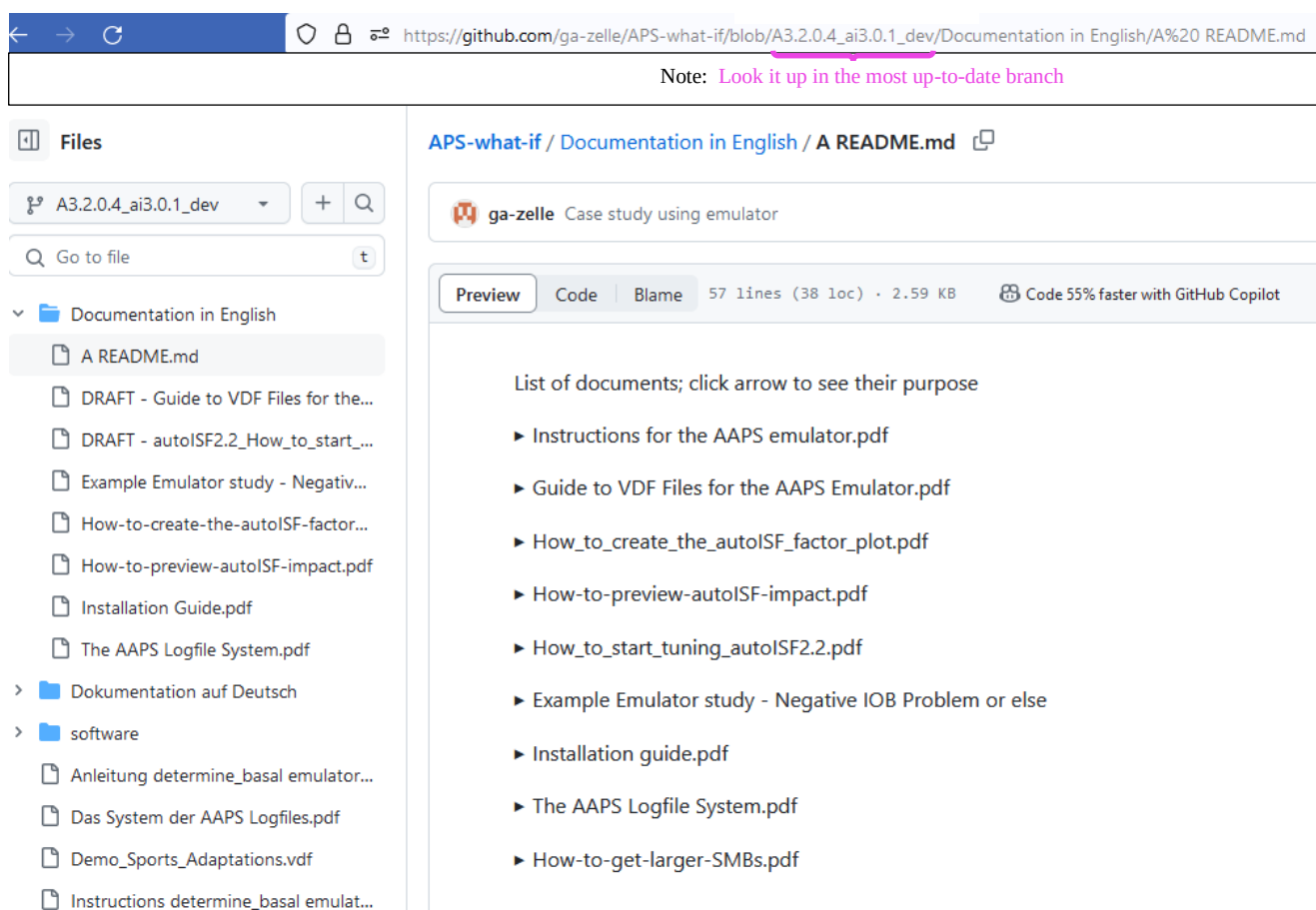
40 More elegant and precise tuning can be done:

- 41 • In a limited but fairly easy way via a new emulator approach by @faldor20:

42 Enter “ aaps-emulator.pages.dev “ in your web browser (Section or Case Study on this welcome!)

- 43 • Note that the iOS based variants of autoISF for Trio or iAPS (oref loops for i-Phone) can not
44 use the emulator. However, the Trio or iAPS integrate on your i-Phone w very basic table
45 that you can look up to see the contributions of the various autoISF categories to the past
46 couple of loop decisions. (More see [section 11.3](#))

- 47 • For Android users, deep analysis is enabled when using a special evaluation software for
48 the AAPS logfiles, the **emulator**. It is described here: [https://github.com/ga-zelle/APS-](https://github.com/ga-zelle/APS-what-if)
49 [what-if](#) / Documentation-in-English. There (under / Software) you find the files needed to
50 download on your PC, and the primary instructions:



51 In the emulator, you can see in tabular and graphical form, which autoISF component, and other
52 settings, contributed to SMB values that determined the glucose curve.

53 In the following, we look into how you create your relevant data.

54 Application examples for tuning are given in associated case studies (we need newer ones).

55

Join <https://discord.gg/n3tD5eXExC> for seeking (and giving) help with the emulator set-up or use, and to exchange experience.

10.1 Installation of the emulator on your PC

Installation is a one-time process, and you best refer to the installation guide of the developer, here:

[Instructions determine basal emulator](#)_AAP3_ai226.pdf (or newer version) in:

<https://github.com/ga-zelle/APS-what-if>

Note that the emulator might not be up-to-date with a recent version release of autoISF. Check directly in Github (link 2 lines above), or see discussion here: <https://discord.gg/n3tD5eXExC>

Below, I attempt to spell out some additional details “for IT dummies” (like myself)

10.1.1 Create your PC folder structure

The suggested folder names and structure shown below is of course not mandatory, but only a suggestion.

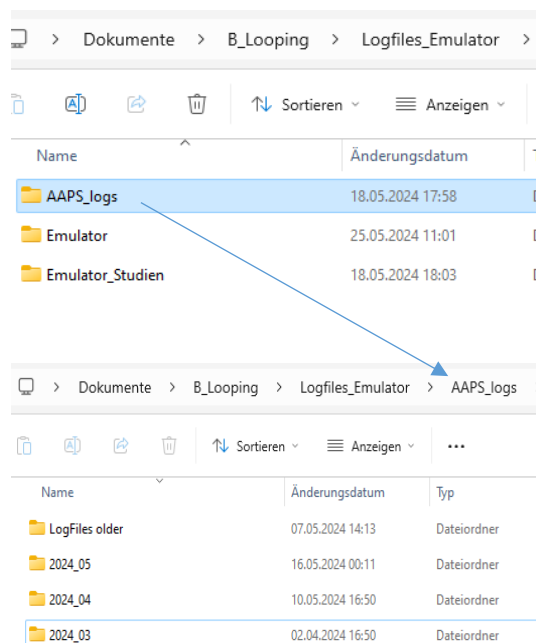
On your PC, create a folder “**Logfiles_Emulator**” with 3 sub-folders: “AAPS_logs”, “Emulator” and “Emulator_Studies”

AAPS_logs: Put all your stored AAPS logfiles into that sub-folder. My folder structure for Logfiles and Emulation on the PC has 3 monthly folders, plus one folder with data from previous months and years (which I am less likely to analyze).

The logfiles you ALWAYS must copy-in from your phone before they get automatically erased there after x days (about 2 weeks, much shorter for 1-minute Libre3).

It is advisable to additionally store a pdf from **Nightscout Reporter** in the file for every month, with daily glucose charts, 24h scatter graph, etc. From it, you can much easier find which days and times are of high interest to analyze with the emulator.

Emulator: Neighboring “AAPS_logs” is the “Emulator” folder into which most downloads from the developer’s repo will go in [section 10.1.2](#)



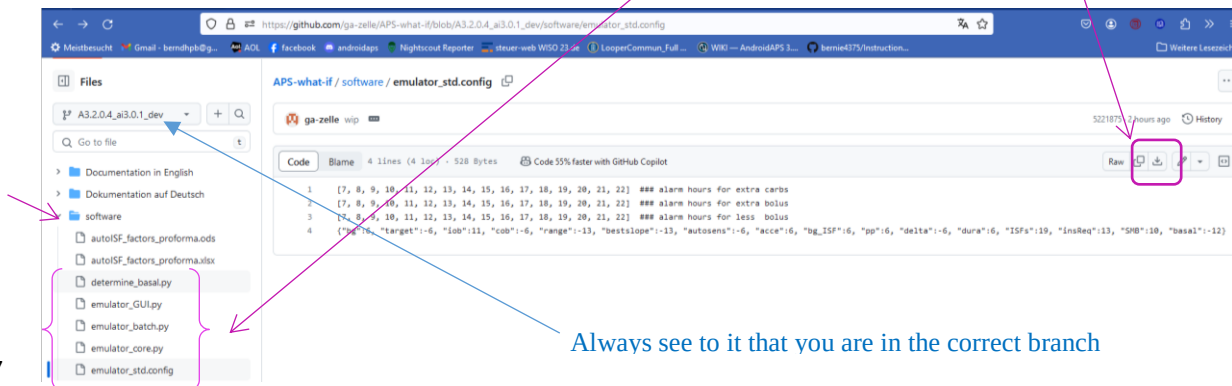
89 **Emulator_studies** is a folder, where, for now, you should provide some **sub-folders** “Study_1”,
 90 “Study_2” ... **Study_n**. Later, when you use the emulator, you will use these “addresses” for the program to
 91 dump results from the emulation into. Additionally you will probably put related AAPS screenshots and
 92 Nightscout.Reporter or xDrip/Statistics charts into each project folder to support analysis.

93

94 10.1.2 Downloads

95 1).Download from: <https://github.com/ga-zelle/APS-what-if/> **software**, the **.config** and four **py**. files.

96 To do this, you must (5 times, one at a time): click on the name here, and, for download, here



Always see to it that you are in the correct branch

97 Always make sure you use the files from the branch with the same version number as your AAPS version (in
 98 the example above: These files will work with AAPS dev version 3.2.0.4 with autoISF version 3.0.1).

99 Always keep your AAPS x autoISF and also the emulator related files up-to-date. If you can't get your
 100 Emulator run, look in the Github repo whether there is a newer .py file (even with the same name; there may be
 101 updates that iron out problems that may have been reported only with certain AndroidOS versions etc etc))!!

102 2).Retrieve these 5 downloaded files on your **PC (list of recent downloads)**, then:
 103

104

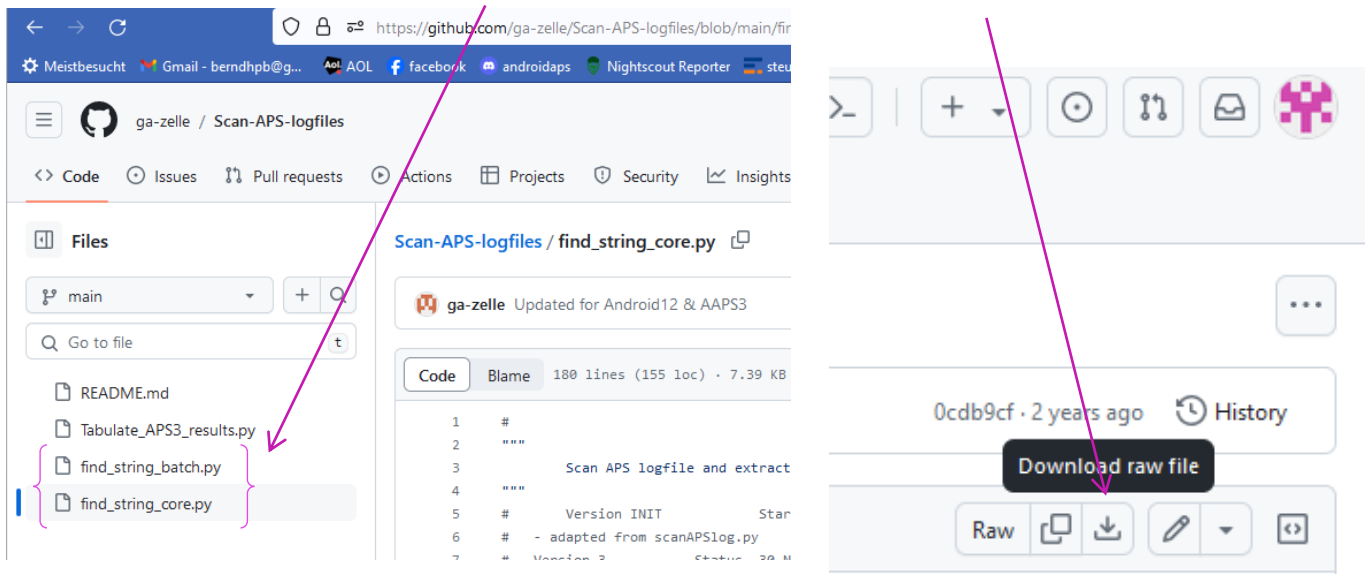
105 3). Shift each of these 5 into your “Emulator” folder:
 106

107

Dokumente > B_Looping > Logfiles_Emulator > Emulator >				
Sortieren Anzeigen ...				
Name	Änderungsdatum	Typ	Größe	
__pycache__	24.05.2024 10:05	Dateiordner		
5minute_emulator_std.config	24.05.2024 22:06	CONFIG-Datei	1 KB	
determine_basal.py	17.05.2024 17:14	Python File	149 KB	
emulator_batch.py	24.05.2024 22:06	Python File	21 KB	
emulator_core.py	24.05.2024 22:06	Python File	164 KB	
emulator_GUI.py	07.05.2024 20:04	Python File	42 KB	
find_string_batch.py	07.05.2024 19:31	Python File	6 KB	
find_string_core.py	07.05.2024 19:31	Python File	8 KB	see 10.1.2 .4)

108 Note: Use 1minute:emulator_std.config in case you use Libre3 (1 min) as your CGM
 109
 110

111 4). From another section in Github, “Scan-APS-logfiles”, fetch two more .py files by
112 repeat steps 1)-3). for these two. They are from: <https://github.com/ga-zelle/Scan-APS-logfiles/blob/main>



113
114
115
116 5)-Retrieve these two .py files in your PC’s downloads folder, and move them into your emulator file (as
117 already was included two pictures higher up).

119 10.1.3 Create an “emulation start button” on your desktop

120 One of the files in your “Emulator” folder is “emulator_GUI.py“

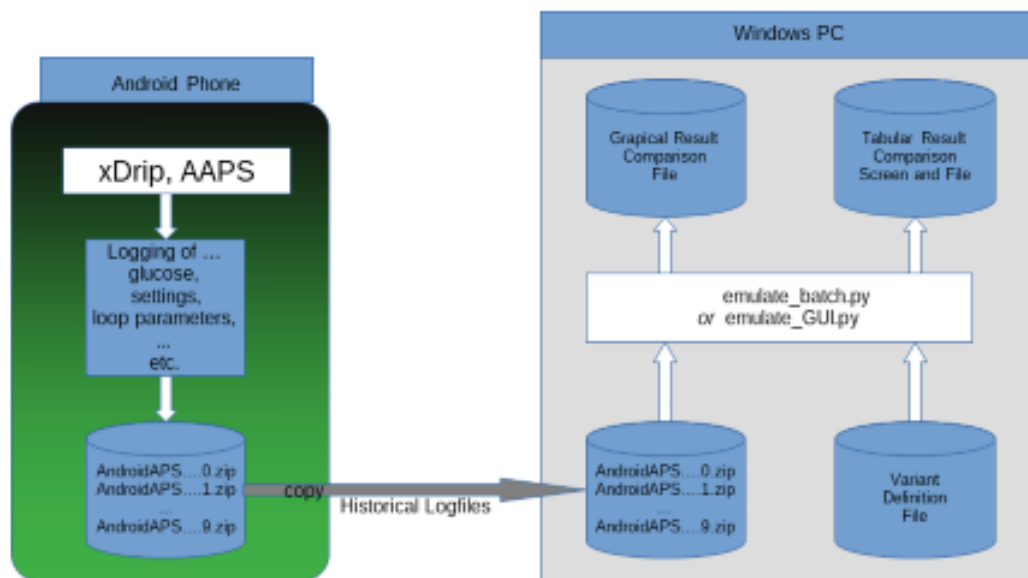
- 121 • Create, in your Emulator folder, a **link to it**
- 122 • Drag **that link** onto your **desktop**
- 123 • Name it something like “Emulator_start”: This is your **start button** for emulations on the PC

125 10.1.4 Other software requirements

126 Make sure you have **Notepad++** on your PC (see [section 10.2.1](#)).
127 QPython 3L will be needed on the smartphone, later ([see section 11](#)).

130 10.2 Analyzing loop decisions in logfiles

131 Instead of making many screenshots every 5 (or, w/ Libre3, every 1) minutes after meals, and analyzing them
132 later, a much more elegant and powerful way to analyze your loop decisions (and how you might want to
133 influence them with different settings, see [section 10.3](#) for this), is to use the emulator.



Sketch of Running the Emulator on a Windows PC

Github/ga-zelle /
APS-what-if

10.2.1 Set up a “no change” .vdf file.

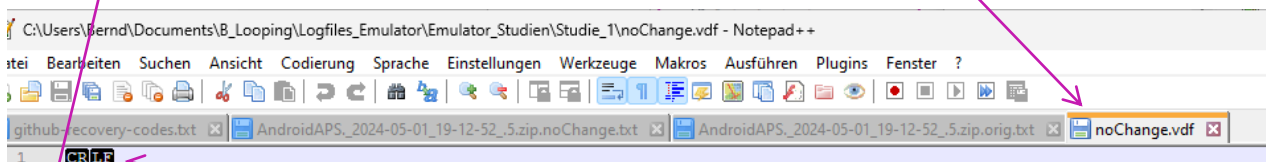
1). To do this, just open **Notepad++** (from list of all programs on your PC).

2). Name your file “noChange.vdf”.

It is just empty in the lines that would define any change to be investigated.

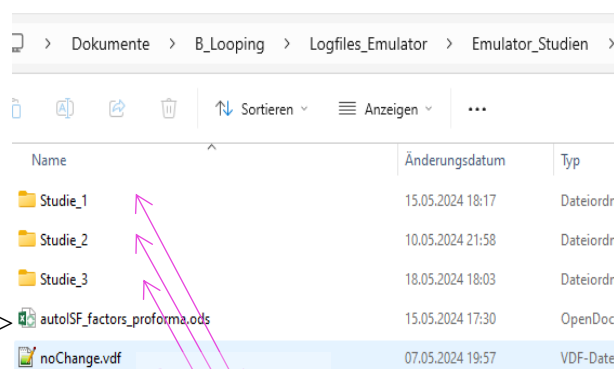
Note: for “what-if” analysis, entries will be made (in a second .vdf later, see [section 10.3](#))

The no change .vdf should look like something like this:



Lines 1 and following should all be free (CR LF might show, depending on your settings in your WORD program)

3). Store that “noChange.vdf” in your “Emulator studies” folder, on the top level, besides the single studies folders



4). From that position, you always make a copy, and paste *into each* Studie_1 ...n :

See [section 10.3.3.3](#), regarding this ->

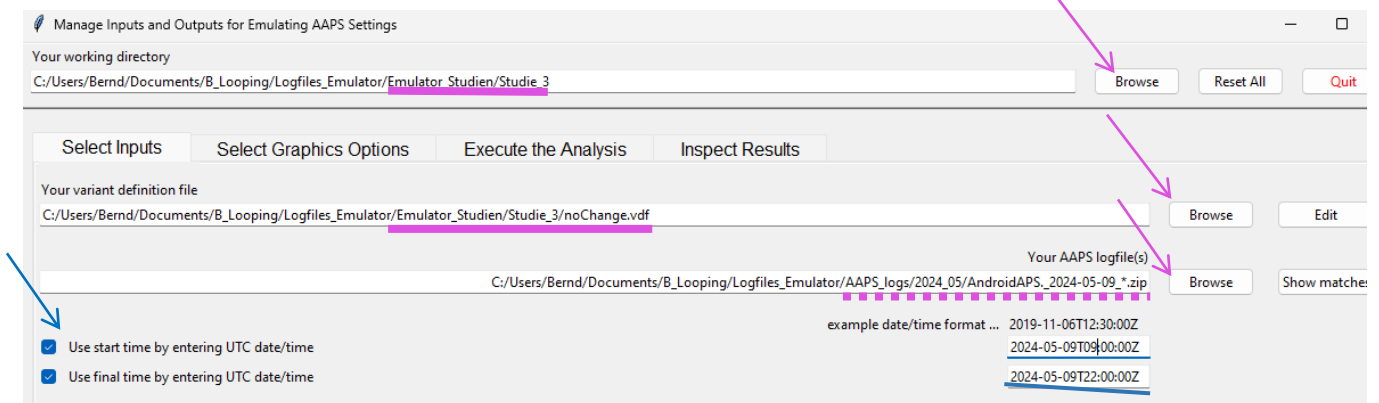
10.2.2 Locate relevant logfiles and prepare the Study_n folder

- 1). Make sure you have the AAPS logfiles that you want to analyze in your “AAPS_logs folder”
- 2). In your “Emulator_Studies” folder, create (or use a prepared) “Study_n” sub-folder, with a copied-in (not: moved!) noChange.vdf (It must be *in all* Study_n files).

10.2.3 Prepare your emulator run for Study_n

Now go to your PC **desktop**, and start the emulator by **just pressing the button “emulator start”** that you installed in step [10.1.3](#)

This opens a big dialogue box with 3 fields that you must fill in with the applicable path (*without* any quotation marks “.”) from your Windows Explorer file system, best done via (3x) Browse button:



- a) The top box marks the path to your current emulator project (“Studie_3” is my “Study_n” where I want to store results)
- b) The middle box marks the path to your current vdf (what kind of analysis; here: “...noChange.vdf” = *read-only*. (For *what-if*, see [section 10.3](#))
- c) The third box marks the path to your AAPS logfiles you wish to look into. A good way to do this is:
 - Browse in your Windows Explorer to any logfile from the desired day (2024-05-09 in above example)
 - Replace the time with an asterix * (this means you look at **all-day** data, in UTZ time). Check whether this will work by pressing Show matches . You should see all logfiles from that day in a pop-up info box.
 - As I wanted to look at 11 am –midnight (for lunch and dinner related data), I :
 - clicked the bottom left two boxes
 - copied the date 2024-05-09 over the default date in the bottom right two data fields

- after T (for time), I entered the desired time of analysis AFTER conversion into my local time (Central EU summer time minus 2 hours = UTZ; so to look at 11 to midnight of my AAPS screen, I must enter here 09.00:00Z, and below it 22:00:00Z).

Entries at the bottom are not mandatory, but when clicking these little boxes (bottom left) you can define a start and/or an end-point for analyzing, within the logfiles specified in the field above.

10.2.4 Run emulation

Now we are ready to go: Press “Run emulation”

This produces sometimes an error message (e.g. if you have a syntax error, or incompatible software versions: => seek help, in the Github materials provided by ga-zelle, or in Discord/Full-Closed-Looping/emulate-aaps here: <https://discord.gg/n3tD5eXExC>

After a short moment results should show up, which you can look into in a couple of ways.

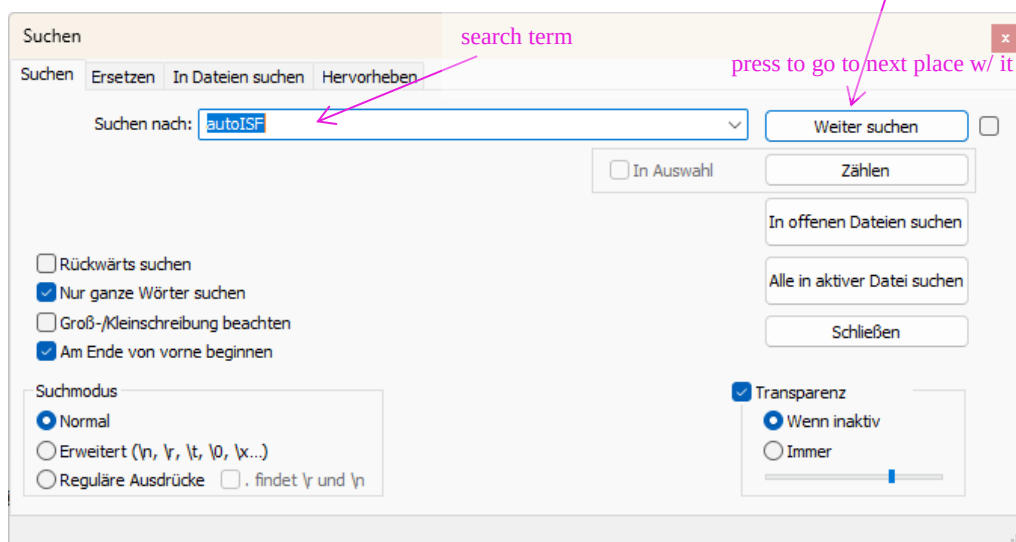
Optional: You could have a quick look into the **.log** file to see whether the run had errors (see [section 3.8](#))

10.2.4.1 AUTO ISF tab contents in (date..) **noChange.txt** result file

This ...txt file basically gives you “all the AUTO ISF tab” infos, in a super long list (but without needing to make screenshots in real-time, every 5 minutes.)

Search options help find what lines are of interest to your analysis:

By using the **search function** you can jump, in that super long list, to all places that e.g. have „autoISF“ in it or „script debug“, or „SMB disabled“ (if you want to analyze when that happened). Precise spelling, as in this .txt (or in AUTO ISF tab) is of course important.



212 10.2.4.2 Table of results (...noChange.csv file)

213

214 The .csv file in your project folder gives a tabular presentation of how parameters like bg, iob,
215 iobTH, the various ISF contributors, bg target, insulinRequired etc. develop every 5 minutes, and
216 what SMB size and %TBR resulted.

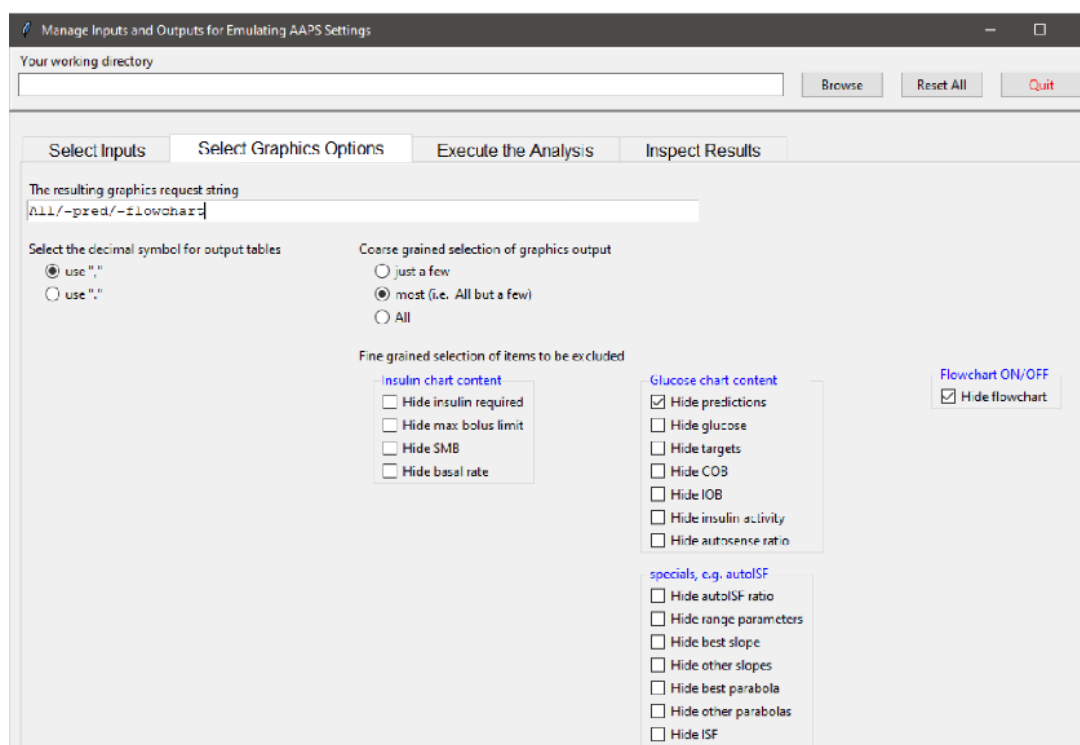
217 It is a vast table, so you may want to reduce it to something more “digestible”, either after transfer
218 to your standard calculation program (next [section 10.2.4.3](#)). Or you can also make settings to suppress
219 information you are usually not interested in (or do not know how to interpret, anyways) under “Select
220 Graphics Options” when you open the emulator, before executing any analysis:

221 First, select your preferred way of outputting decimals (point or comma).

222 Then select whether you want “All” possible outputs in the graph, or “Most” = all except those you tick “off”
223 in the boxes for each output parameter.

224 In case you would use “Some/just a few”, you would have to tick those few you that do want to see, by ticking
225 the corresponding boxes.

226 Recommendation is to look at (nearly) everything offered (as your default setting that you can leave
227 untouched in most of your emulator runs):



228

229 It might be easier, to not deal with customizing the csv file, and rather copy the data into your
230 favorite calculation program:

231

232 10.2.4.3 Analysis of the noChange.csv table in Excel or LibreOffice calc.

233

234 Best copy the entire table into a new .xls or .ods sheet, where you can:

- 235
- add right next to the UTC (Unix Time Code) your corresponding “AAPS time”

236 For instance, adding +2/24 translates the UTC column into central European summer time column
 237 next to it (where currently a row of Z stands). Likewise, subtract like -5/24 from UTC for an US East
 238 Coast time scale.

239 *(Fun fact: Our oref loop stubbornly works on UTC, un-impressed by our folly to jump twice a year*
 240 *into or out of a local summer time, or to travel across time zones. If some data get lost in translation*
 241 *there, it is only to us, with our stupid time change. For the loop, its database (e.g. on insulin activity)*
 242 *remains unambiguously intact).*

243 Highlight all time fields (both entire columns), and switch from hh:mm:ss format to hh:mm.
 244 *(While the seconds are important for the loop's calculations, for our comparison with Nightscout or*
 245 *other charts and data, it is much easier without the seconds attached)*

246 • **hide** any column you find less important to look at for your intended analysis

247 That way, “boxes” (data fields) retain their original position in tables

248 Also, in case later you want to look into additional info, you can simply un-hide the relevant columns
 249 (or lines:.)

250 • **hide** lines (time segments) you find less important to look at for your intended analysis

251

252 Usually you will color mark where relevant SMBs were given, which of the ISFs (and underlying
 253 weights) was strongly contributing (note that this can be good or not good). Also where iobTH was
 254 exceeded, whether an Automation kicked in e.g. setting a TT, or when there were periods with zero
 255 insulinRequired.

256 In [section 10.3.4](#) we present an extra tool that does a standardized table reduction and color marking
 257 for you!

258 You may be able to formulate a hypothesis or two, what settings (...ISF_weights, iobTH%,
 259 SMB_range_extention, autoISFmax ...) should be changed for improvement (then go to [10.3](#))

260

261 [10.2.4.4.. Graph noChange.pdf](#)

262

263 After your emulation run, under Inspect Results, you can open the pdf file that is last in the results list
 264 offered.

265

266 This **noChange.pdf** is a chart that shows along the time axis (down), from right to left:

267 • Red: the bg curve

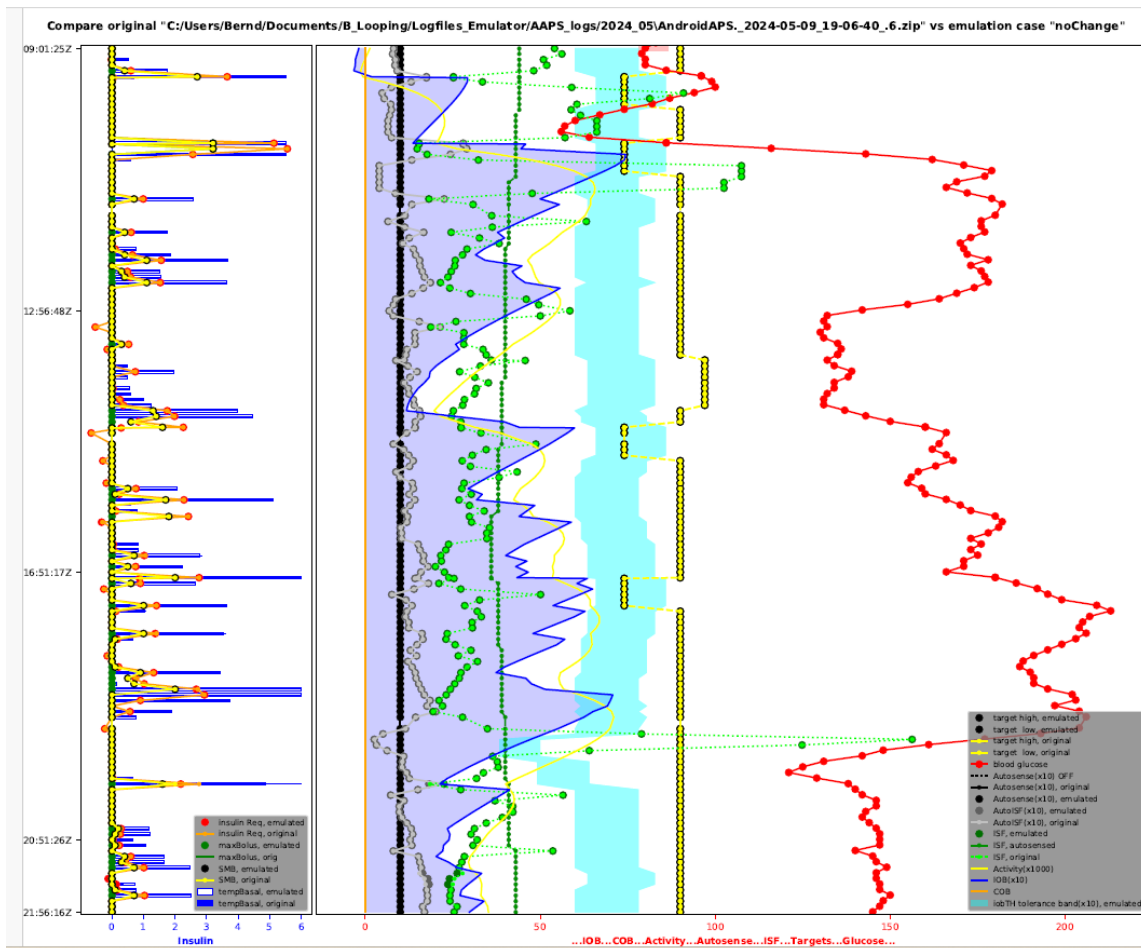
268 • Yellow: the bg target (note that I do no manual “EatingSoonTT” but for bg rises over +10 mg/dl
 269 I have an Automation that sets low TT for a couple of minutes)

270 • Light blue corridor: Left edge is set iobTH, and bandwidth +30% (would be +20% at elevated
 271 TT)

- Dark blue line: iob (exceeding twice the iobTH, with temp. SMB shut-off)

As bg did not convincingly come down enough, one could hypothesize that iobTH should be elevated. ((But, again, this would have to be confirmed also with other kinds of meals)).

- Thin yellow line: Insulin activity
- Green dotted line: ISF as would result from AAPS w/Autosens
- Green scatter points: autoISF ISF no Chage (lighter points) or what-if (darker points)
- Black line: Profile ISF
- Gray scatter points: ISF weakened (to the left of black line) or strengthened (to the right)
- Orange line: cob=0 at all times (in FCL)



More see discussed together with (yourChanges).pdf in [section 10.3.3.4](#)

10.2.4.5 delta table

In case you want to analyze delta, short and long average deltas:

- you could do some of that just using your .xls extract from .csv (see section 10.2.4.3).

- There is also an extra sheet provided, “delta” (That works only if you have your **emulator_core.py** updated to the one from 02June2024 or newer).
- This is definitely core to FCL using Automations ([section 13.1](#)).
- When using acceleration detection via autoISF, the deltas are mainly meaningful to define personal Automations that involve Conditions using these glucose curve characteristics. An example would be to use the delta table to identify cases of compression low

10.3 “What-if” analysis using the emulator

In the following you see an example how you can analyze a day of logfiles, and selecting the time span of interest, for instance 11-24 h to look at how autoISF managed lunch and dinner.

You will go through the emulator exactly as you already did in [section 10.2](#), where you exclusively had the noChange.vdf on board.

However, this time you focus on (yourChange).vdf, see below, [10.3.1](#).

Repeat, if you have two or more such vdf defined.

(Just clear old results before pressing “execute analysis” each time.)

All results are automatically captured for all runs, all **in your selected “Study_n” folder**, together with the noChange results

- Results files with noChange in their name are always your actual loop data ...
- as opposed to results on “what-if”, that contain name of the (yourChange).vdf in their file name see e.g. at around [line 380](#)

How to proceed, step by step:

10.3.1 Define your investigated changes in (yourChange).vdf (one, or several)

1). Define for which parameter(s) in your current profile settings you want to look into a different setting. Recommendation is to use a factor, like for example: current setting * 0.9, or current setting * 1.2, and use that in your naming for this vdf file, too.

You may want to consult [APS-what-if / Documentation in English / Guide to VDF Files for the AAPS Emulator.pdf](#) Access directly, or via [section 3.8](#)

323 **Within one study, you can make several emulator runs with several (yourChange).vdf files** (all based on
324 what really happened, as captured with the noChange.vdf).

325 All results, like the csv results table, will appear then *several times* in your study file, only *with different*
326 *name endings* as in the underlying vdf.

327

328 Example: I like to check in my actual data (they are in my noChange.vdf emulator run), **in which time**
329 **points the following parameter changes would make a** (how) big **difference** in the loop's decision:

330 • 20% higher bgAccel_ISF_weight to boost the first SMBs stronger: How would that tend to ramp
331 up early iob; and might that get too strong in other parts of the data? Or does it bounce into a
332 restriction (maxSMB size; autoISFmax; iobTH...) that I might need to widen?

333 • Doubling my cautiously set bgBrake_ISF_weight shall give me insight into the workings of that
334 parameter (and whether using a much smaller weight than for bgAccel_ISF_weight is really
335 what I should keep doing)

336 • As my bg came down from a persistent high quite slowly, I elevate the dura_ISF by 20%

337

338 *Tuning advice:* Actually, it would make more sense to first find my “optimal”, maybe indeed
339 elevated, bgAccel_ISF_weight. *Then*, in a *new* project_n+1, do (automatically) a noChange run
340 **with that**, plus a (yourChange) run with the stronger dura weight, investigated *on that* basis.
341 Reason: 1) As we always say, better do only one change at a time. 2) A better job with bg control
342 via bgAccel_ISF will reduce the peak height and provide a different (easier) scenario for
343 dura_ISF to manage.

344

345 2).Now, to **write your (yourChange). vdf for the emulator** (this is same procedure as you did in section
346 10.2.1 for the noChange.vdf):

347

348 • just open Notepad++ (from list of all programs on your PC) to create a new vdf:.

349 Alternatively you can also take another pre-existing vdf file, copy it into your current
350 project , edit as desired, and give it a new name (re-name it)

351 **Caution:** Make absolutely sure (best by looking it up in the AUTO ISF tab, down in the profile set
352 section) to **spell each term exactly** as your loop uses it (probably w/ decimal points, not comma)

353 2.1) Investigating simple **setting changes by a factor** multiplied with the former value

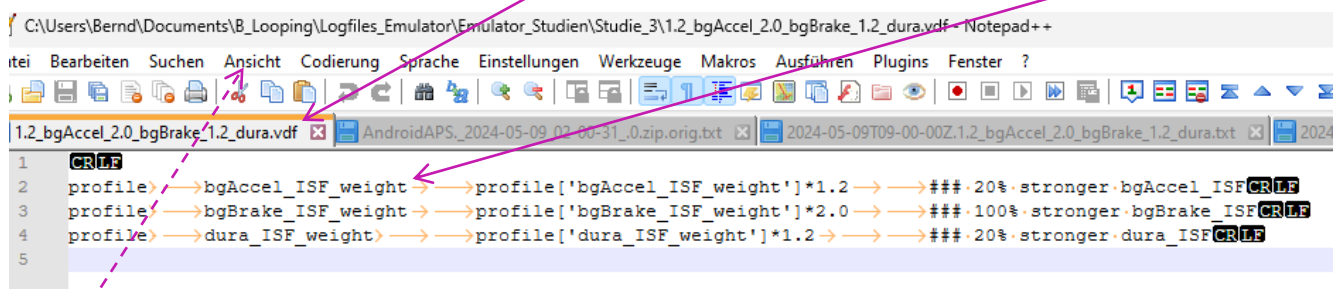
354 • ...when you make one line per parameter (separating entries with spacers->):

355 profile->(parameter) ->->profile['(parameter)']*(factor)->->###(comment as you like)

356

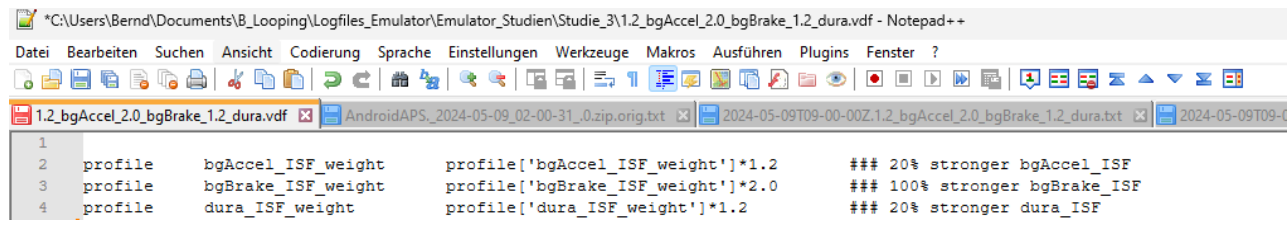
357

The (yourChange) .vdf should look like something like this:



Via view/ show/hide symbols (CR , LF, tabs ...), you can have different looks, see other example two pages down.

Or, with settings/view/remove symbols (for line feed, tab etc) it can also look like this:



2.2) More complicated vdf files are needed if it is not just a factor applied to a setting.

See [APS-what-if / Documentation in English/Guide to VDF Files for the AAPS Emulator.pdf](#).

One example from there:

In the VDF you can also define your **circadian tables for ISF, CR and basal**. You need to pay special attention to the difference between UTC time and your local time zone for the pump. Here is an example for CET (Central European Time):

STAIR_ISF	00:00:00Z	45	### 01h_	C(entral)	E(uropean)	T(ime) or
02h_	CEST					
STAIR_ISF	01:00:00Z	44	###			
STAIR_ISF	02:00:00Z	42	###			
...						
STAIR_ISF	17:00:00Z	36	### 18h_	CET		
STAIR_ISF	18:00:00Z	38	###			
...						
STAIR_ISF	22:00:00Z	43	### 23h_	CET		
STAIR_ISF	23:00:00Z	44	### 00h_	CET		
profile	sens		STAIR_ISF	###		
STAIR_CR	00:00:00Z	8.0	### 01h_	C(entral)	E(uropean)	T(ime) or
02h_	CEST					
STAIR_CR	01:00:00Z	7.5	###			

```

397 ...
398
399 STAIR_CR          20:00:00Z    7.5      ###
400
401 STAIR_CR          21:00:00Z    8.0      ###
402
403 STAIR_CR          22:00:00Z    9.0      ### 23h_CET
404
405 STAIR_CR          23:00:00Z    9.0      ### 00h_CET
406
407 profile    carb_ratio          STAIR_CR    ###
408
409
410
411 STAIR_BAS          00:00:00Z    0.41      ### 01h_C(entral)E(uropean)T(ime) or
412 02h_CEST
413 STAIR_BAS          01:00:00Z    0.43      ###
414
415 STAIR_BAS          02:00:00Z    0.44      ###
416
417 STAIR_BAS          03:00:00Z    0.50      ###
418
419 ...
420
421 STAIR_BAS          19:00:00Z    0.75      ### 20h_CET
422
423 STAIR_BAS          20:00:00Z    0.75      ###
424
425 STAIR_BAS          21:00:00Z    0.60      ###
426
427 STAIR_BAS          22:00:00Z    0.45      ### 23h_CET
428
429 STAIR_BAS          23:00:00Z    0.43      ### 00h_CET
430
431 profile    current_basal        STAIR_BAS    ###
432
433

```

434 Some lines were omitted in that example for better readability. The lines must be sorted by UTC
 435 time. Therefore the first line (winter time) or first two lines (summer time) from your CET/CEST
 436 based pump definition must be cut off and appended at the end. If your profile is not fully populated
 437 for 24 hours this may also mean you first need to create a pump entry at 01 or 02 hours,
 438 respectively.

439 To have such a vdf file ready is a great idea for investigations into changes of your profile:
 440 See [section 11.4](#) for how to get a voice literally tell you, whenever the new profile would
 441 actually make a difference, and you having the choice to watch performance with ‘) or
 442 without that change (* only if the new profile makes more aggressive settings)..

443
 444 3). Name your vdf (in example below: 1.2_bgAccel_2.0_bgBrake_1.2_dura.vdf) ...

445
 446 4). Store this (yourChange).vdf in the folder for your current Study_n you are about to start
 447 (see my storage path C: Studie3.....vdf – Notepad++ in the top line:)

448
 449
 450
 451

10.3.2 Run the emulator with (yourChange).vdf

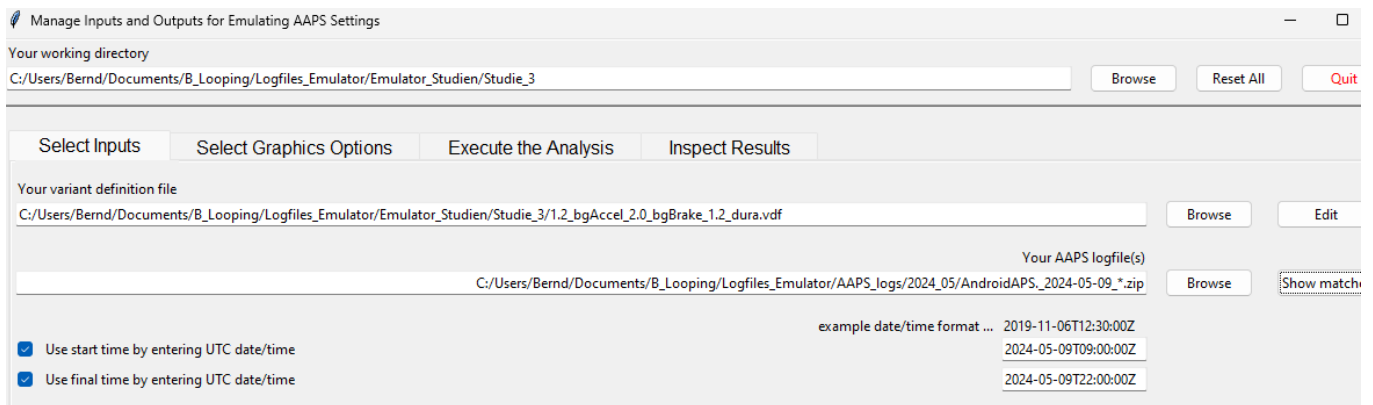
453

454 The “what-if” emulator run is done the same way as you did the noChange.vdf run ([section 10.2](#)), which had
455 no (yourChange).vdf on board

456 => No surprise, running an emulation with only that noChange.vdf, yields same result in emul columns as is
457 orig columns. - However, now :

458

459 The **(yourChange).vdf** must be loaded into the 2nd input field, where formerly you had the noChange.vdf.:



460

461

462 In the 3rd input field, give the path to your stored logfiles. A good way to do this is:

- 463
- Browse in your Windows Explorer to any logfile from the desired day (2024-05-09 in above example)

464

 - Replace the time with an asterisk * (this means you look at all-day data, in UTZ time). Check whether this will work by pressing Show matches . You should see all logfiles from that day in a pop-up info box.

465

 - As I wanted to look at 11 am –midnight for lunch and dinner related data, I :

466

 - clicked the bottom left two boxes

467

 - copied the date 2024-05-09 over the default date in the bottom right two data fields

468

 - after T (for time), I entered the desired time of analysis AFTER conversion into my local time (Central EU summer time minus 2 hours = UTZ; so to look at 11 to midnight of my AAPS screen, I must enter here 09.00:00Z, and below it 22:00:00Z).

469

470

471

472

473

474 After making these entries, press Execute the Analysis, (evtl also Clear old Data) and then press Run

475 Emulation, you can look the results up under “Inspect Results”. First you could have a quick look into

476 the .log file to see whether the run had errors (see [section 3](#).)

477

478

479

480

481

482 10.3.3 Emulation results
483

Your working directory
C:/Users/Bernd/Documents/B_Looping/Logfiles_Emulator/Emulator_Studien/Studie_3 Browse Reset All Quit

Select Inputs Select Graphics Options Execute the Analysis Inspect Results

*.log - Your file showing edits from the variant assignments
C:/Users/Bernd/Documents/B_Looping/Logfiles_Emulator/Emulator_Studien/Studie_3/2024-05-09T09-00-00Z.1.2_bgAccel_2.0_bgBrake_1.2_dura.log Browse Show

*.csv - Your table comparing key values of original vs emulation
C:/Users/Bernd/Documents/B_Looping/Logfiles_Emulator/Emulator_Studien/Studie_3/2024-05-09T09-00-00Z.1.2_bgAccel_2.0_bgBrake_1.2_dura.csv Browse Show

*.delta - Your table comparing bg deltas of original vs emulation
C:/Users/Bernd/Documents/B_Looping/Logfiles_Emulator/Emulator_Studien/Studie_3/2024-05-09T09-00-00Z.1.2_bgAccel_2.0_bgBrake_1.2_dura.delta Browse Show

*.orig.txt - Your short log of original analysis
C:/Users/Bernd/Documents/B_Looping/Logfiles_Emulator/Emulator_Studien/Studie_3/2024-05-09T09-00-00Z.orig.txt Browse Show

*.txt - Your short log of emulated analysis
C:/Users/Bernd/Documents/B_Looping/Logfiles_Emulator/Emulator_Studien/Studie_3/2024-05-09T09-00-00Z.1.2_bgAccel_2.0_bgBrake_1.2_dura.txt Browse Show

*.pdf etc. - Your graphic file comparing key values of original vs emulation
C:/Users/Bernd/Documents/B_Looping/Logfiles_Emulator/Emulator_Studien/Studie_3/2024-05-09T09-00-00Z.1.2_bgAccel_2.0_bgBrake_1.2_dura.pdf Browse Show

484
485 All results from your (yourChanges).vdf emulator go automatically where the noChange.vdf results are
486 already stored, in our example into the “Studie 3” file, below:

487
488 Besides the 1.2_bgAccel_2.0_bgBrake_1.2_dura.vdf case which I like to look into for the present high carb
489 meal, I also prepared another vdf that investigates a factor 1.2 stronger pp_ISF and a weaker, factor 0.8,
490 bgAccel_ISF (with the intention to test this, and a noChange (that ideally would already contain the conclusion
491 on adapting the bgAccel_ISF_weight*), on a low carb meal later.

492 * Note the challenge here is to iterate between the typical meals of your personal spectrum to find **one** set
493 of settings that work good-enough **for all** of them.

Studie_3 durchsuchen

Name	Änderungsdatum	Typ	Größe
2024-05-09T09-00-00Z .noChange.pdf	15.05.2024 17:17	Adobe Acrobat-D...	77 KB
2024-05-09T09-00-00Z .noChange.csv	15.05.2024 17:17	Microsoft Excel C...	51 KB
2024-05-09T09-00-00Z .noChange.log	15.05.2024 17:17	Notepad++ Docu...	35 KB
2024-05-09T09-00-00Z .noChange.txt	15.05.2024 17:17	Notepad++ Docu...	281 KB
2024-05-09T09-00-00Z .orig.txt	15.05.2024 17:17	Notepad++ Docu...	281 KB
2024-05-09T09-00-00Z .1.2_pp_0.8_bgAccel.pdf	15.05.2024 17:16	Adobe Acrobat-D...	78 KB
2024-05-09T09-00-00Z .1.2_pp_0.8_bgAccel.csv	15.05.2024 17:16	Microsoft Excel C...	51 KB
2024-05-09T09-00-00Z .1.2_pp_0.8_bgAccel.log	15.05.2024 17:16	Notepad++ Docu...	57 KB
2024-05-09T09-00-00Z .1.2_pp_0.8_bgAccel.txt	15.05.2024 17:16	Notepad++ Docu...	281 KB
2024-05-09T09-00-00Z.1.2_bgAccel_2.0_bgBrake_1.2_dura.csv	17.05.2024 21:29	Microsoft Excel C...	51 KB
2024-05-09T09-00-00Z.1.2_bgAccel_2.0_bgBrake_1.2_dura.log	17.05.2024 21:29	Notepad++ Docu...	66 KB
2024-05-09T09-00-00Z.1.2_bgAccel_2.0_bgBrake_1.2_dura.pdf	17.05.2024 20:40	Adobe Acrobat-D...	78 KB
2024-05-09T09-00-00Z.1.2_bgAccel_2.0_bgBrake_1.2_dura.txt	17.05.2024 21:29	Notepad++ Docu...	282 KB
1.2_bgAccel_2.0_bgBrake_1.2_dura.vdf	17.05.2024 20:38	VDF-Datei	1 KB

494

495

496

 1.2_pp_0.8_bgAccel.vdf	10.05.2024 21:55	VDF-Datei	1 KB
 noChange.vdf	07.05.2024 19:57	VDF-Datei	1 KB

10.3.3.1....(yourChange).txt: “what-if” impact on loop decisions (as in AUTO ISF tab)

The **noChange.txt** has all the info your series of AUTO ISF tabs had that day.

How to search in this vast list is shown elsewhere (see [section 10.2.4.3](#)).

Likewise, the **(yourChange).txt** gives *for each loop decision* in all detail how and why each single decision *would have* changed with the different parameter inputs you are checking out here

In the two (yourChange) examples here, , it was a check on the difference

- a 20% stronger pp_weight and 20% weaker bgAccel_weight
- a 20% stronger weight for both, bgAccel_ and dura_ISF, and a doubling of bgBrake_weight

would make.

Note that all these “what if” data can only give rough hints, notably about **the first** greater change that you would see with the investigated changed setting. So it works quite well for our main problem in FCL, investigating how to ramp up iob quickly after detection of acceleration.

Note that any relevant change would put your bg curve on a different trajectory, so that would influence *all following* results. Therefore, what you get here is **not** a complete modelling how your bg would have developed in the alternative scenario.

But you can investigate in which stages the parameter(s) you are looking at in your current “what-if” had big influence, and in which direction the changes would go. (see also charts shown in [section 10.3.3.4](#)).

Analyzing how to safely come down from a high glucose plateau while limiting hypo danger towards the end of digestion is also to some extent possible.

A good other way to employ the what-if analysis is real time, on your smartphone, using speech synthesis (see [section 11](#)): Then you get real-time info, as to exactly when a significantly different proposal would emerge, and can decide (and watch!) real-time whether to follow the new idea and not was probably better.

Observe that a setting change must work well for you

- not just in one point of time, and
- not just for one kind of meal,

but you must look at all time slots in the investigated meal, plus analyze with the same tool a totally different meal within your usual spectrum, how things work out there

533 10.3.3.2 Tabular results

534

535 A) .csv results table and spreadsheet copies of it

536

537 The **noChange.csv** table gives all relevant data. Besides development of bg and iob you see the calculated
538 insulinRequired in each loop decision, and how each of the autoISF categories contributed to the decision
539 (notably regarding SMB size).

540

541 Note that the “**acce_ISF**” results are only in case of positive acceleration (that is our main focus)
542 driven by the bgAccel_ISF_weight setting. (These are all positions > 1.0 in the “acce ISF” columns).

543

544 **In case of negative acceleration** (decelerating rise, positions < 1.0 in the “**acce ISF**” columns),
545 **bgBrake_ISF_weight is applied**. As discussed in [section 4.4](#), bgBrake_ISF might be most
546 important (and interesting to analyze) in slowly resorbing meals.

547

548 Note: maxBolus=0 means in this table that SMBs were not capped by maxBolus.

549

550

551 The **(your change).csv** shows in detail how **every single** loop **decision** would be influenced by the different
552 settings you are investigating.

553 To inspect that huge table, click on the Z behind the start UTC time entry (see black box in the Z column of
554 the table, next page).

555 If you like to see the bg in each screen, too, go 3 or 4 columns farther to the right with your black
556 box.

557 Then, go to window/fix. Now you can scroll through the data and always see headline and time (or time and
558 bg level).

559 To further ease analysis, feel free to temporarily erase (hide) any columns that you (think you) do not
560 need for the intended analysis. More suggestions see in [section 10.2.4.2](#)

2024-05-09T09-00-00Z .1.2_pp_0.8_bgAccel.csv - LibreOffice Calc

DateiBearbeitenAnsichtEinfügenFormatExtrasDatenFensterHilfe

Liberation Sans10

</

Still, the csv tables are overwhelming. You could proceed in either of two directions now:

- A) Convert both (or all 3) csv files into one table in Excel or into Libre office calculator. Hide columns (and eventually also lines) that are of no particular interest for your analysis. Mark differences between noChange and (yourChanges) column data with color, add extra columns with additional calculations ...

This route is good to compare quantitative impacts of autoISF categories in critical time points.

- B) For the core data relevant to assessing your autoISF settings, there is an extra tool for convenient analysis - see the following [section 10.3.3.3](#)

That tool is quite a bit of work to set up. Decide for yourself whether you do it, or whether you rather work with extracting the csv table into Excel (A), and work freely from there.

574 10.3.3.3 Automated extraction from tabular results (optional add-on)

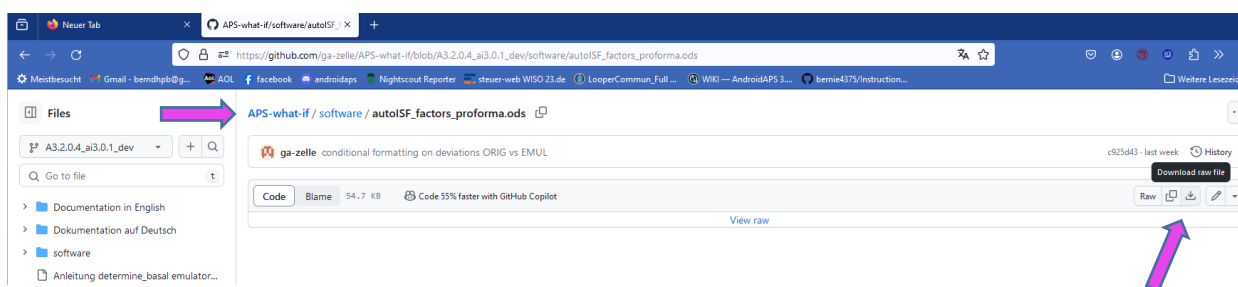
575 Decide for yourself, whether you rather go from the csv results table into .xls and produce what you want to
576 see there for yourself.

577 With a bit of extra set-up effort (steps 1)-7), next 4 pages) you can install an adjunct tool that will always pro-
578 duce the nice graph for you as shown on the end of this section 10.3.3.3, in step 7).

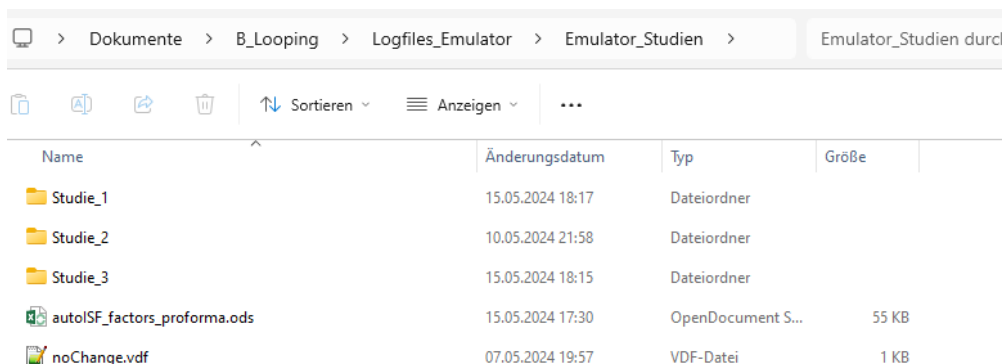
579 Realizing that going through the following steps 1) – 7) is currently quite complicated, the
580 developer is working at radically simplifying it in an update to be expected this summer
581 (2024). If you feel a bit challenged, we suggest to just wait for this update, which will largely
be embedded in a newer version of the autoISF_factors_proforma.ods, see next section.

582 1).autoISF_factors_proforma.ods is provided as an extra tool that you download from here:

583 www.github/ga-zelle/APS-what-if/software



584
585 Put that file on your PC one level above the single files for all your studies:
586
587



588
589 2).Now, if we want to use this tool on the two csv files of our Studie_3 file, we must proceed as follows (for
590 each of the two .csv files, separately):

- 591
- 592 1. Click on the .csv file and open in Libre office calculator.
 - 593 2. Make sure the time column is set to US_English:

Textimport - [2024-05-09T09-00-00Z%20.1.2_pp_0.8_bgAccel.csv]

Importieren

Zeichensatz: Unicode (UTF-8)

Sprache: Standard - Deutsch (Deutschland)

Ab Zeile: 1

Trennoptionen

☐ Feste Breite ☒ Getrennt

☒ Tabulator ☐ Komma ☒ Semikolon ☐ Leerzeichen ☐ Andere

☐ Feldtrenner zusammenfassen Texttrenner: "

Weitere Optionen

☐ Werte in Hochkomma als Text ☒ Erweiterte Zahlenerkennung

Feldbefehle

Spaltentyp: US-Englisch

	Standard	US-Englisch	Standard	Standard	Standard	Standard	Standard	S
1		UTC		UNIX	accel	bg	target	
2		time		time			low	
3	id						orig	
4	0	09:01:25 Z		1715245285,9	80		90	
5	1	09:06:20 Z		1715245580,3	79	79	90	
6	2	09:11:24 Z		1715245884,2	80		90	
7	3	09:16:26 Z		1715246186,2	80		90	
8	4	09:21:21 Z		1715246482,0	86		90	

Hilfe OK Abbrechen

594

595

596 3).Now start, in Libre office calculator, the autoISF_factors_proforma.ods ...

597 This turns the first 30-some lines of your csv table (left side) into a form in which important effects are

598 highlighted in color, and formatting is improved:



599

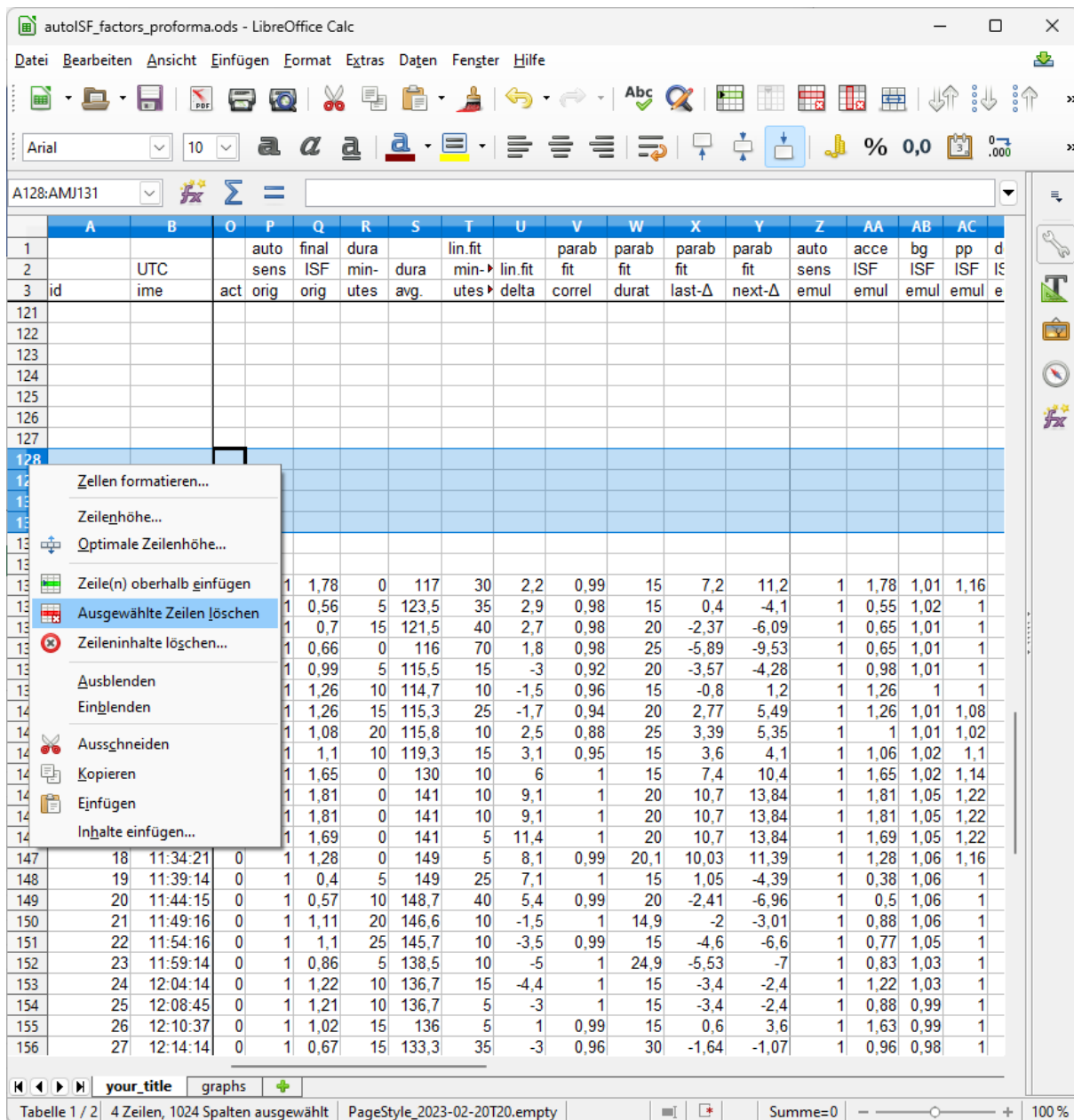
600

601 Now, you want this for the entire table.

602 4).In the autoISF_factors_proforma table, highlight 20 or more lines (not including the first or last), and
603 mouse right hand/insert above ...

The screenshot shows the LibreOffice Calc application window titled 'autoISF_factors_proforma.ods'. The spreadsheet contains a table with columns labeled A through AC. A right-click context menu is open over cell A34, displaying options such as 'Zellen formatieren...', 'Zeilenhöhe...', 'Optimale Zeilenhöhe...', 'Zeile(n) oberhalb einfügen', 'Ausgewählte Zeilen löschen', 'Zeileninhalte löschen...', 'Ausblenden', 'Einblenden', 'Ausschneiden', 'Kopieren', 'Einfügen', and 'Inhalte einfügen...'. The table data includes columns for 'id', 'ime', 'UTC', 'act', 'orig', 'sens', 'ISF', 'min-utes', 'dura', 'avg', 'lin.fit', 'min-utes', 'lin.fit', 'parab', 'parab', 'parab', 'parab', 'auto', 'acce', 'bg', 'pp', 'd', and 'e'. The status bar at the bottom indicates 'Tabelle 1 / 2 | 29 Zeilen, 1024 Spalten ausgewählt | PageStyle_2023-02-20T20.empty | Summe=49196174579,56'.

604
605
606 Do this as often as you need to create the number of lines that your emulated csv file comes with.
607 If you ended up with too many lines, erase the superfluous number (any four, in the example):



608
609

610 5). Then just copy it in, by selecting all data lines in the emulated csv, and pasting (paste special, values only)
611 into box A4 of your “elonged” autoISF_factors_performa.ods.

612

613 6). The bottom tab “your_title” should be re-named by you, best with day of log you analyze, and your what-
614 if parameters (so, the name of your csv file could be put in here)

615

616 Now you have a table with optimized lay-out that incorporates key data from both your no change
617 AND of your investigated changes.csv files.

618

619 A similar table is available on the (i-)phone if you use the autoISF dev variant of iAPS or Trio (see
620 [section 11.3](#))

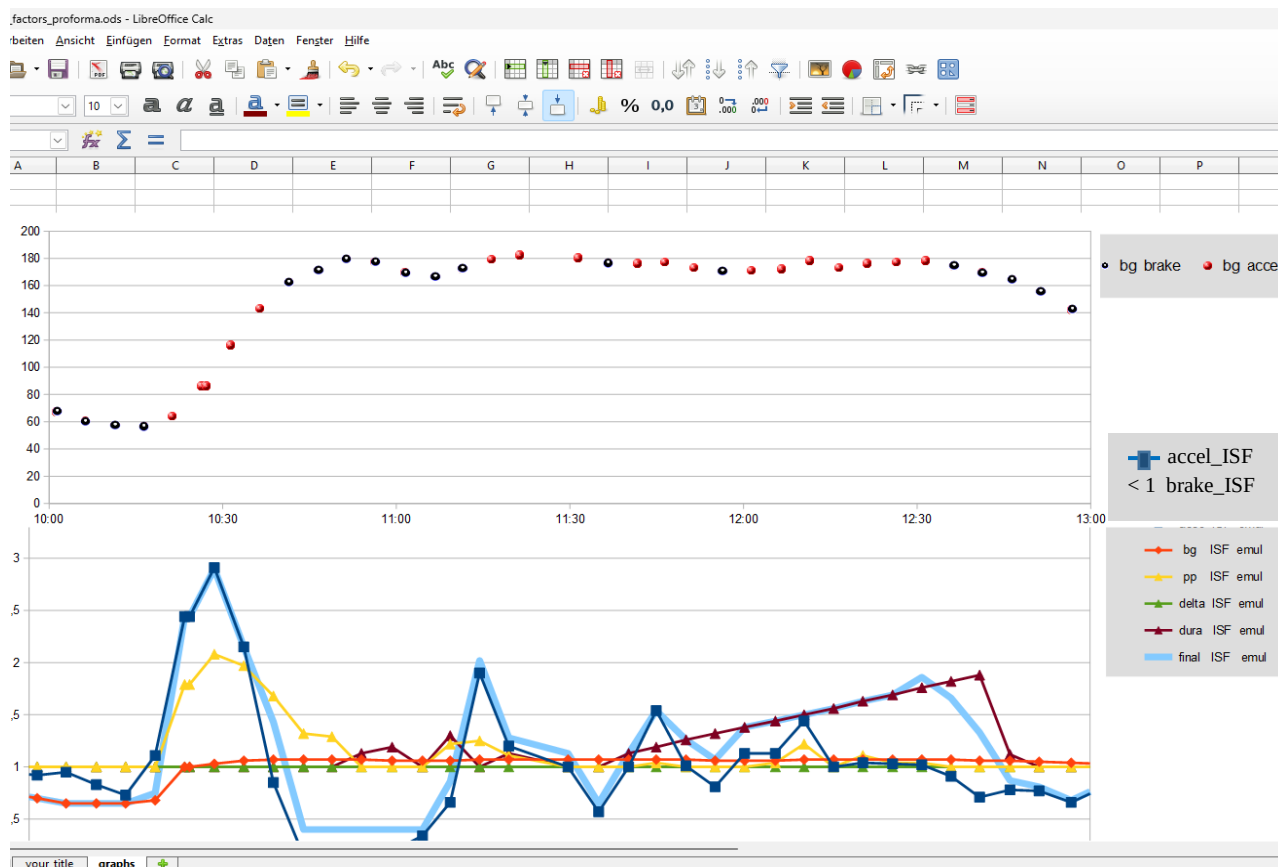
621

622 7).A super neat extra feature is already pre-programmed, which you can see if you click on the bottom **tab**
623 **“graphs”**.

624

625 The top graph is the bg curve (the actually seen bg).

626 Note that for the what-if no bg development over the time range is available. (The noChange one is
627 also given there).

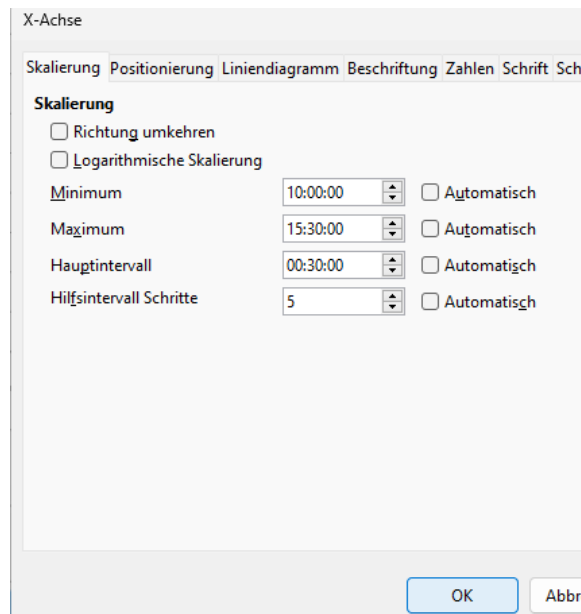


628
629

630 The bottom graph (do one for each, the noChange or the (yourChanges) case) shows the amplification factors
631 coming from each autoISF category, and the overall resulting ISF amplification.

632

633 You probably have to widen the time scale (double click on the time axis, and type the desired time
634 span (min and max UTC)(and spacing of data points, 00:30:00 or 00:15:00) into this box:



In the given example above, the 2.5 hours were not enough yet to analyze this 10:30 UTC (12:30 AAPs) lunch; we need to look until bg is near target (hopefully before dinner starts).

10.3.3.4 Chart coming with the emulator

In case you find the extra steps described in the preceding section “too much”: Also the emulator offers one chart (the pdf offered at the bottom of the screen as shown below the “[10.3.3](#) Emulaton results” headline).

First look at the initial bg rise in the noChange.pdf chart (emulation results from your noChange.vdf run), and see how bgAccel_ISF and pp_ISF acted, or could have acted in improved ways.

Then look into in (yourChange).pdf to see potential effects (or what other change to try). (Actually, you probably will have to go into a detailed analysis of several lines and columns of the tables as discussed in sections [10.3.3.2](#) and [10.3.3.3](#)).

Note that ideally we want FCL coverage of our entire “normal day” meal spectrum by **one** set of settings.

So, not much is gained if you put a lot of effort in optimizing FCL settings for one meal.

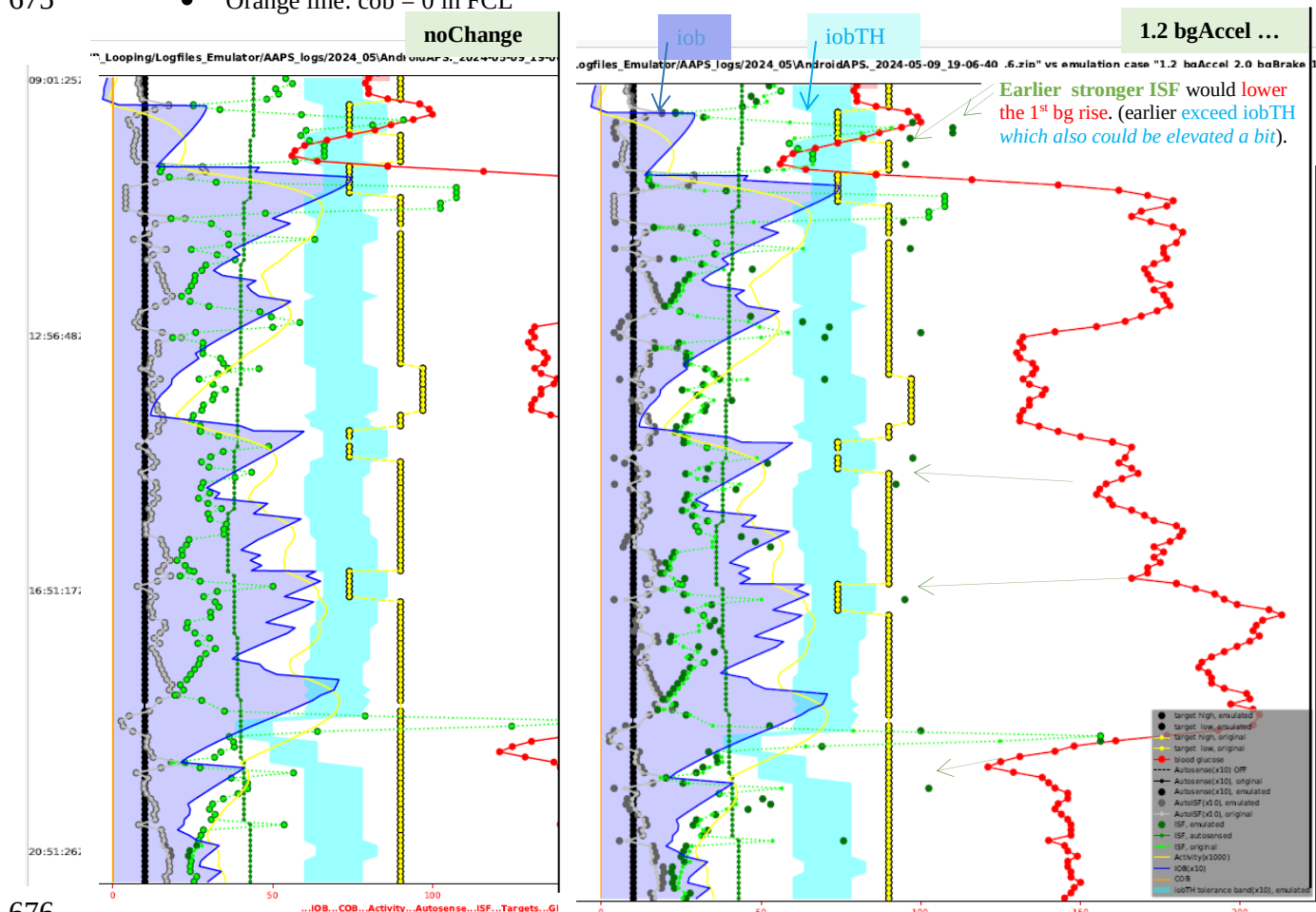
You will need iterations. Do such analysis for **two or three very different meals** that you wish the algorithm to automatically handle. See [section 4.2/4.3](#) on how meals with very different carb loads might benefit or also suffer from aggressive or mild (category)_ISF_weights you could set.

The initial iob received might be limited by allowed SMB sizes, autoISFmax, or the (dynamic!) iobTH. You will have to look into the data table to find out about this (a quick orientation - notably regarding the light blue iobTH band, see next page - is also possible in the pdf result files you have in your project file (project file example “Studie 3” in 2nd chart under the [10.3.3](#). headline).

662 Only once you found OK weights for bgAccel- and pp_ISF_weights, does it make sense to go tune the
 663 dura_ISF_weight. 12:00 – 12:45 UTC in above graph, the resulting effective ISF is dominated by dura_ISF.
 664 Just judging from the picture, a stronger weight might be worth trying. However, we really need to see the
 665 insulinRequired calculation and the further development because impatience about bringing bg values down
 666 faster too often results in hypoglycemia later.

667
 668 The **noChange.pdf** is a chart that shows along the time axis (down), from right to left:

- 669
- Red: the bg curve
 - Yellow: the bg target (note that I do no manual “EatingSoonTT” but for bg rises over +10 mg/dl I have an Automation that sets low TT for a couple of minutes)
 - Light blue corridor: Left edge is set iobTH, and bandwidth +30% (would be +20% at elevated TT)
 - Dark blue line: iob (exceeding twice the iobTH, with temp. SMB shut-off)
 - Orange line: cob = 0 in FCL
- 670
671
672
673
674
675



- 676
677
- As bg did not convincingly come down enough, one could hypothesize that iobTH should be elevated. ((But, again, this would have to be confirmed also with other kinds of meals)).
 - Thin yellow line: Insulin activity
 - Green dotted line: ISF as would result from AAPS w/Autosens
- 678
679
680
681

682 • **Green scatter points:** autoISF ISF no Change (lighter points) or **what-if (darker points)**

683 Foreseeably, this is the strongest difference between our noChange (left) and 120% bgAcel_ISF_weight
684 (right) in the picture below. (Note the red bg curve is *both times* the really seen bg, because the what-if
685 case only looks at each single loop decision). The first (->) time the dark green dot is far to the right, this
686 would get the bg down, we would start to see a (<-) bg lowering effect, shifting the red curve to the left

687 • Black line: Profile ISF

688 • Gray scatter points: ISF weakened (to the left of black line) or strengthened (to the right)

689

690 Regarding the other changed parameters: Stronger dura_ISF would suggest more insulin towards the end of
691 plateaus; this should have helped in the 1st plateau (red curve, top right quadrant of the picture). However,
692 same setting would have to work also on 2nd plateau; the chart cuts off there, so too early to see whether a
693 hypo danger might result.

694 Effect from doubling the bgBrake_ISF effect are hard to evaluate. Better probably to look in .csv tables, or
695 run a separate emulation for that change only.

696

697 Always check for 2 or 3 kinds of your meals whether the “new” parameter settings really are on average
698 better. (See negative example in [case study 8.2!](#))

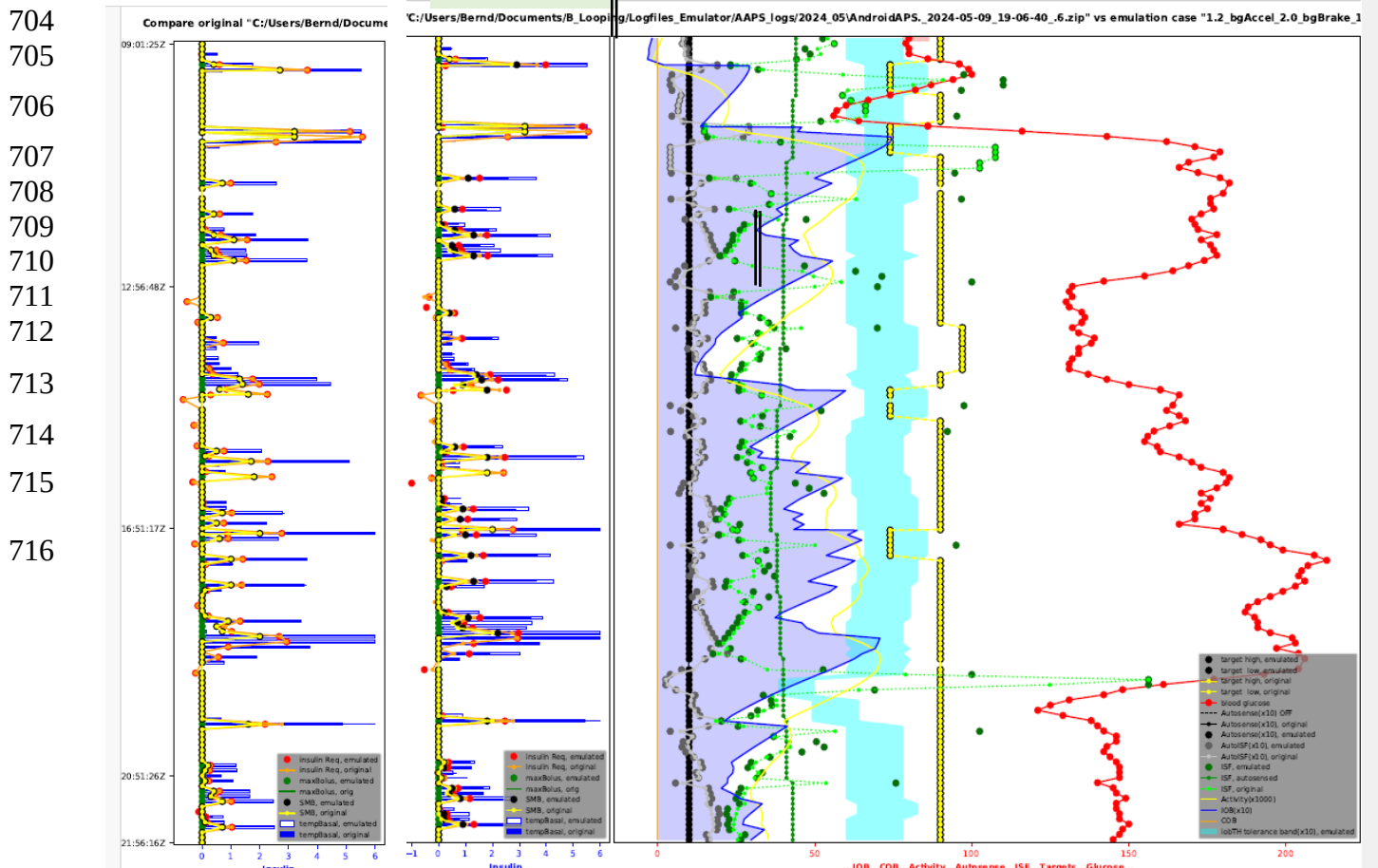
699

700 Part of both above shown charts (left side of each, with blue peaks) was cut out.....

701 (unfinished / to be explained later) (Here: yourChange = 1.2_bgAcel_2.0_bgBrake_1.2_dura)

702

703 noChange || 1.2 bgAcel ...



717 10.3.3.5 delta table from “what-if” run with (yourChange).vdf

718

719 In case you want to analyze delta, short and long average deltas, see [section 10.2.4.5](#)

720

721 To analyze deltas in a “what-if” scenario really does not make much sense, because effects from

722 each single change ripples through many subsequent situations, and it is impossible to predict how

723 glucose curve, and therefore also how deltas, would develop in the what-if case.

724

725